LifeRouter
Technical Manual

Kentalives

June 2026

Contents

Chapter 1

Technical Manual

This manual contains the information needed to modify, test, and maintain LifeRouter. The user manual describes installation, configuration, and use of the public API. This document focuses on the internal code structure, package dependencies, service lifecycle, and the points where developers must be careful before introducing changes.

1.1 Internal System Organization

The project is a Go module that combines a public API for integrations, an executable entry point for running the service as a standalone process, and an `internal/` tree reserved for implementation code. This separation matters because external consumers can import public packages, while internal packages must only be used by this module.

Path	Main responsibility
`main.go`	Standalone process entry point. It loads configuration, creates the default dependencies, runs the dispatcher, and waits for system shutdown signals.
`embedded/`	Public surface for running the service inside another Go process. It exposes default configuration, default dependency wiring, and the startup call that delegates to the same application path as the binary.
`pkg/config`	Public configuration and dependency-injection types. Changes to these types can break external repositories that build custom configurations.
`pkg/domain`	Exported domain types: costs, visualization states, emergency directions, registered NATS-domain errors, and geospatial query or publishing interfaces.
`pkg/pathfinding` and `pkg/emergency`	Go clients over NATS for starting agent routes, controlling communicators, and managing emergency routing. They do not own the NATS connection they receive.
`pkg/subjects`	Public constants for the NATS subjects handled by the service. They allow direct NATS use without duplicating string literals in Go consumers.
`pkg/snapshotstream`	Shared codec for per-floor snapshots sent through NATS streams. It preserves the simple shape for small messages and chunks large snapshots with `gzip+json`.
`pkg/tuning`	Public helper tools for inspecting rasterized maps and tuning the grid before running the service, including textual output and a browser visualizer for large grids.

Table 1.1: Executable entry point and public API responsibilities.

Path	Main responsibility
`internal/app`	Application wiring, world initialization, NATS dispatcher, request handlers, active-agent state, and emergency lifecycle.
`internal/config`	Configuration loading with Viper, default values, and process-wide storage for configuration and dependencies used by internal packages.
`internal/grid` and `internal/raster`	Grid representation, coordinates, multi-floor world, portals, and conversion from map images into traversal costs.
`internal/pathfinding`	Algorithm implementations: shared primitives, dynamic agent planning, and emergency flow fields.
`internal/mock`, `internal/pathfinding/testing`, and `internal/integration`	Test doubles, shared scenarios, and global behavior tests for the service.
`data/`	Map images used as inputs in tests and rasterization tuning.

Table 1.2: Internal implementation packages and supporting data.

1.1.1 Boundary Between Public API and Implementation

The public boundary is formed by `embedded/` and `pkg/`. The `embedded` package lets another Go application run the service in its own process through `DefaultConfig`, `DefaultDependencies`, and `Run`. This path is useful for integrations, tools, and debugging runs, but it still requires a NATS connection and domain dependencies that are coherent with the real environment.

The `pkg/` directory contains the types and clients that other repositories can use. The `pkg/pathfinding` and `pkg/emergency` packages translate Go calls into NATS requests. `pkg/config` defines configuration and injectable dependencies. `pkg/domain` contains shared types such as costs, directions, preferred-route graphs, and domain errors. `pkg/subjects` exposes NATS subject constants for consumers that build messages directly. When this surface changes, both internal tests and possible external integrations must be considered.

By contrast, `internal/` must not be treated as a stable API. It contains global application state, algorithms, and communication details. A change inside `internal/` may legitimately avoid external compatibility, but it must preserve the behavior observed through the public API and NATS subjects.

1.1.2 Service Lifecycle

The standalone process and the embedded mode use the same application path. The main difference is who creates the configuration, dependencies, and cancellation context.

Phase	Technical description
Initial load	`main.go` creates a context cancellable by `SIGINT` or `SIGTERM`, calls `config.LoadConfig`, and obtains values from defaults, `config/config.yaml`, and environment variables.
Default dependencies	`embedded.DefaultDependencies` opens separate NATS connection pools for geospatial queries and publishing, and prepares a simulated external system. Production deployments can provide their own dependencies through `pkg/config.Dependencies`.
Application startup	`app.Run` stores global configuration and dependencies, initializes logging, builds the rasterized world with layers and portals, creates the service NATS connection, and registers handlers.
Normal execution	The `Dispatcher` keeps active agent communicators, briefly retains completed agent results, and manages the current emergency state. Blocking request paths use worker limits to avoid unbounded goroutine growth.
Graceful shutdown	`Dispatcher.Shutdown` stops active agents, stops the emergency if one is running, drains the service NATS connection, cancels the service context, and closes dependency resources that expose `Close`.

Table 1.3: Main lifecycle phases of the microservice.

1.1.3 Dependency Model

Internal packages access configuration and dependencies through `internal/config`. This simplifies access from algorithms and handlers, but it requires service startup to call `config.Setup` before code that reads `config.Cfg` or `config.Dep`. Tests that exercise internal packages must explicitly prepare this state when they call functions that depend on it.

Element	Use inside the system
`pkg/config.Config`	Complete configuration: NATS connection, logging, grid dimensions, layers, portals, agent parameters, emergency parameters, and geotruth pool sizes.
`pkg/config.Dependencies`	Groups the external systems required for domain data, geospatial reads, and movement publishing.
`domain.ExternalSystem`	Provides information not owned by pathfinding: object costs, floor heights, and emergency signaling nodes.
`domain.GeoQuery`	Read-side geospatial interface: nearby objects, object data, oriented object sets, and floor/region lookup for a real-world point.
`domain.GeoPublish`	Write-side interface used to publish an agent's new position and receive the external commit acknowledgement.

Table 1.4: Configuration and external dependencies used by the application.

1.1.4 Geotruth Connection Pools

The default dependencies wrap `GeoQuery` and `GeoPublish` in internal NATS client pools. The query pool is used for object data, nearby objects, oriented objects, and floor resolution. The publish pool is used for position updates. Separating both flows avoids making local agent vision and real-position publishing always compete for the same NATS connection.

Pool sizes are configured under `pathfinding.geotruth` through `queryconnections` and `publishconnections`. If omitted, both default to `GOMAXPROCS`; values lower than one are normalized to one connection. Client selection uses round-robin atomic counters. During shutdown, `Dispatcher.Shutdown` closes dependencies that implement `Close`, including the pools created by `DefaultDependencies`.

1.1.5 NATS Handler Ownership

The concrete NATS subjects are documented in the user manual and defined as public constants in `pkg/subjects`. For maintenance, the important point is which dispatcher area owns each handler group and which internal state it changes.

Group	Internal responsibility
Agent pathfinding	Starts planning for an existing object, converts the goal from meters to grid coordinates, creates an internal `AgentCommunicator`, and replaces any previous run for the same agent.
Agent path observation	Associates a state-publishing channel with the active communicator and streams snapshots until the route ends or the communicator closes the channel.
Agent communicator control	Checks movement state, waits for completion, changes speed, requests discrete or meter-based movement, pauses automatic movement, or terminates the route.
Emergency pathfinding	Applies preferred routes to the global world, converts goals to grid coordinates, starts the flow field, and prevents simultaneous emergencies.
Emergency flow observation	Publishes per-floor direction maps while an emergency is active and closes the subscription when the flow publisher finishes.
Emergency stop and shutdown	Coordinates emergency shutdown, waits for completion, and restores global costs when preferred routes are removed.

Table 1.5: NATS handler groups and associated internal state.

1.1.6 Validation Points for Changes

Changes to public APIs or NATS messages must be reviewed together with the user manual because they alter the integration contract. Changes to the dispatcher or lifecycle should run the `internal/app` tests and integration tests. Changes to grid logic, rasterization, or algorithms require the corresponding unit tests and, if performance is affected, the profiling or benchmark tests already separated in the repository.

The general validation command is:

```
make test
```

During local iteration, run the package-level tests first. Before closing a change, run the full suite. When measuring code exercised by all tests across packages, use the full coverage target described in the testing section.

1.2 Spatial Representation: Grid, Rasterization, and Multi-Floor World

The spatial representation is shared by agent pathfinding and emergency routing. The service does not work directly on continuous geometry. It works on a per-floor cost grid. Public API coordinates arrive in meters; before algorithms run, those coordinates are

converted to cells and associated with a world layer.

1.2.1 Units and Coordinates

Internal local coordinates use `grid.Coords`, which contains integer `X` and `Y` cell indexes. Conversion between meters and cells depends on `GridConfig.CellSizeM`. Cell-to-meter conversion returns the cell center. Meter-to-cell conversion rounds toward the corresponding cell and never returns negative coordinates.

Element	Behavior to preserve
`CellSizeM`	Real-world side length represented by one cell. It must be greater than zero; invalid global configuration causes the grid package to panic.
`Coords.ToFloat64`	Converts cell indexes to meters and returns the center of the cell. It is the reference for publishing or comparing real positions derived from the grid.
`CoordsFromFloat64`	Converts an `x,y` meter position to a cell coordinate using active configuration and clamps negative coordinates to zero.
`GlobalCoords`	Adds a layer index to a local coordinate, distinguishing the same `x,y` cell on different floors.
`GlobalCoordsFromFloat64`	Uses `GeoQuery.RegionFromPoint` to resolve the layer and then converts `x,y` to a cell. It requires `config.Dep.Qu` to be initialized.

Table 1.6: Main units and conversions in the spatial representation.

Global coordinates must be validated before converting them to flattened indexes. A cell whose `X` or `Y` is outside its floor cannot rely on `ToIdx`, because arithmetic could alias another floor's index range. The intended exception is the emergency fake goal, represented as a sentinel outside the grid with flattened index `-1`.

1.2.2 Grid Cost Layers

Each `Grid` keeps three same-size buffers. The base layer comes from the rasterized map and floor masks. The object layer represents dynamic costs, such as obstacles or observed agents. The final layer is the effective cost read by the algorithms. This separation keeps the base map stable while updating only regions affected by dynamic information.

Concept	Behavior to preserve
`base`	Permanent floor costs. A zero value blocks a cell and must not be overwritten by ordinary dynamic updates.
`objects`	Temporary costs added by objects or local updates. A zero-cost object region can clear previous temporary costs and restore the base cost.
`final`	Cost used by pathfinding. It is recomputed with `AddCost(base, objects)` and change notifications are emitted when it changes.
`RegionGrid`	Rectangular region relative to a parent grid. It is used to replace object costs, apply base masks, and limit updates around a position.
`SubChanges`	Registers synchronous callbacks for batches of changed cells. Updates must preserve notification of previous costs.
`AddCost` and `DiagonalCost`	Cost operations that are safe around unreachable values and overflow. Algorithms must use these helpers rather than adding costs directly.

Table 1.7: Internal grid layers and related operations.

Main mutation points are object-region replacement, base-mask application, final-cost recomputation, and base reset. Any change here must preserve blocked cells, publish each batch of changes once, and keep base, object, and final layers synchronized. A future extension could page the local object and final layers, similar to the sparse storage already used for D* Lite values.

1.2.3 Map Rasterization

World creation calls `grid.FromImg`, which builds each floor base layer through `raster.RasterPNG4WithAura`. `pxpercell` tells how many source image pixels correspond to one grid cell. Traversable cells are selected from the red channel: in simple paletted images, the red palette entry is used; in generic images, the average red value must pass the internal threshold. Wall aura then increases cost around blocked cells using a Chebyshev neighborhood.

Change point	Main risk
Image format handling	Changing palette detection or fallback behavior can make existing maps non-traversable or unexpectedly expensive.
Pixel-to-cell sampling	Changing center sampling or scaling rules can move walls and corridors relative to the configured grid.
Wall aura	Changing the Chebyshev radius or cost can close narrow corridors or remove clearance around walls.
Base-cost convention	Changing empty-space and unreachable values affects all grid and pathfinding cost calculations.

Table 1.8: Rasterization risks that require explicit validation.

1.2.4 Multi-Floor World, Portals, and Virtual Worlds

`World` groups floor grids and stores metadata that flattens global coordinates into unique indexes. Layer names are translated to internal indexes through the configured `GridConfig.FloorLayers`. Geotruth region names must match these configured names when public `[x,y,z]` coordinates are converted into layers.

Portals are directed cross-floor edges. Global portals are configured during startup. Local portals belong only to a leased virtual world and are useful for agent-specific changes. The world stores portals indexed by origin and by destination, so `PortalsFrom` and `PortalsTo` do not need to scan unrelated portals.

Element	Role
`NewWorld`	Rasterizes configured layers, installs the process-wide global world, and later receives configured portals.
`NewVirtualWorld`	Leases a per-agent world that lazily links floors from the global world and keeps local portals separate.
`AddPortal` and `AddBidirectionalPortal`	Add directed or paired global cross-floor edges. Invalid costs, same-floor portals, and invalid endpoints are rejected or logged.
`AddLocalPortal`	Adds a portal visible only in the leased virtual world, preserving the global world for other agents and for emergency routing.
`PortalsFrom` and `PortalsTo`	Merge global and local portals from the corresponding origin or destination indexes.

Table 1.9: Multi-floor world elements and portal responsibilities.

1.2.5 Validation of Spatial Changes

Spatial changes must be validated at the layer they affect: coordinate conversion, grid costs, rasterization, portals, or virtual-world reuse.

Change	Minimum validation
Coordinate conversion	Tests for meter-to-cell and cell-to-meter conversion, negative coordinates, configured cell size, and layer resolution.
Cost-layer mutation	Tests that blocked base cells remain blocked, final costs update correctly, and subscribers receive changed cells.
Rasterization	Tests with representative map fixtures, aura radius, complex-image fallback, and red-channel threshold behavior.
Portals	Unit tests for directed and bidirectional portals, same-floor rejection, incoming-portal lookup, removal, and virtual-world local portals.
Virtual-world reuse	Tests that reused worlds do not leak local portals, subscriptions, or dynamic objects from a previous agent run.

Table 1.10: Recommended validation for spatial changes.

1.3 Agent Pathfinding With D* Lite

Agent routing is based on D* Lite [1]. The planner controls movement, local geotruth vision, dynamic cost updates, and route publication. Each route runs in a goroutine and owns a virtual world derived from the global world.

1.3.1 Lifecycle of an Agent Route

Phase	Description
Request reception	The dispatcher decodes `AgentFindPathParams`, checks shutdown state, converts the goal to `GlobalCoords`, and starts internal agent pathfinding.
Initial state	The agent position is read from geotruth, converted to grid coordinates, and associated with a leased virtual world.
Planning	D* Lite initializes values, priority queue state, and the current goal; it then computes the shortest path from the current position.
Movement loop	While the start is not the goal, the communicator decides whether a step can be taken based on speed, manual commands, and termination signals.
Local update	Nearby objects are queried from geotruth, rasterized into local grid overlays, and changed cells trigger D* Lite repairs.
Real publication	According to `CellsForRealUpdate`, internal position is converted to meters and published with `GeoPublish.UpdateObjectPosition`; floor height is read from `ExternalSystem.HeightAtFloorPoint`.
Completion	The communicator records the terminal error, closes publishers and waiters, releases the virtual world, and the dispatcher retains the result briefly.

Table 1.11: Agent route lifecycle.

1.3.2 D* Lite State

State	Role
`g` and `rhs`	The D* Lite value pair. Dense mode uses one contiguous store; sparse mode allocates value pages only where needed.
Priority queue	Queue with lookup, update, and removal by index. Keys encode D* Lite priority and depend on the current start.
`start` and `goal`	The current position is stored as coordinates; the goal is stored as a flattened global index.
World subscription	Receives changed cells from the virtual world and transforms them into D* Lite vertex updates.
Portal state	Successor and predecessor expansion must include same-floor neighbors and cross-floor portals.

Table 1.12: Main state used by the agent D* Lite planner.

The sentinel index `-1` is reserved for the emergency fake goal and must not be treated as a normal agent pathfinding cell.

1.3.3 Local Vision and Dynamic Changes

Agent replanning is driven by local observations. The planner asks geotruth for nearby objects inside the configured vision radius converted to meters. Object geometry is rasterized into the agent's virtual-world object layers. Changed cells are then propagated to D* Lite, which repairs only the affected parts of the search state. This conversion follows a scanline-style rasterization strategy commonly used to fill geometry over pixel or cell grids [2, 3].

Element	**Meaning**
`VisionRadiusCells`	Radius of local geotruth refresh in grid cells; it is also converted to meters for the external query.
`SparseValuePageDirectory`	Enables sparse storage for D* Lite `g/rhs` values in large local worlds.
Object rasterization	Converts oriented objects into dynamic grid costs using object traversal costs from the external system.
Changed-cell repair	Updates predecessor and successor costs so D* Lite can reuse previous search work instead of planning from scratch.

Table 1.13: Configuration and dependencies used by agent local vision.

1.3.4 Communicator and Movement Control

The `AgentCommunicator` connects public commands, internal route state, and goroutine lifecycle. It must maintain the ordering of commands for one agent; worker limits in NATS handlers do not parallelize one agent's algorithm.

Responsibility	Behavior
`UpdateMovementSpeed`	Changes continuous movement speed in cells per second. A zero value pauses automatic movement without terminating the job.
`MoveNCells`	Requests a bounded number of grid-cell steps.
`MoveFMeters`	Blocks until the requested distance in meters is consumed or the agent exits; it returns remaining meters when the request cannot be fully consumed.
`Stop`	Pauses automatic movement by setting speed to zero.
`Terminate`	Cancels the route and stores `ErrAgentTerminated` if it was still active.
`BlockingWait` and `ExitError`	Allow clients to wait for completion and query the terminal domain error. The dispatcher also supports these calls briefly after completion.
Path publisher	Streams route-visualization snapshots. Replacing a publisher closes the previous one.

Table 1.14: Agent communicator responsibilities.

Automatic movement uses absolute next-tick scheduling. The communicator schedules the next movement instant before returning to route processing, so planning and publication time count against cadence. If a tick is already overdue, the implementation permits one immediate step and then resets timing from the current time rather than catching up with bursts.

1.3.5 Resource Ownership and Validation

An agent run owns its virtual world, local subscriptions, path publisher, wait channels, and terminal result. Changing this ownership can leave callbacks active, stale data in reused worlds, or terminal errors unavailable to clients. Use tests around replacement by a same-agent run, termination, retained results, and shutdown races whenever this area changes.

1.4 Emergency Routing With Flow Fields

Emergency routing computes a global flow field toward one or more exits. The flow field is updated when the environment changes and can be mapped to emergency signaling nodes. Unlike agent routing, this uses the global world because preferred routes intentionally modify global base costs and are restored when the emergency stops.

1.4.1 Emergency Lifecycle

Phase	Behavior
Start request	The dispatcher checks that no emergency is already running, applies preferred routes, converts goals from meters to grid coordinates, and starts the flow field.
Initialization	The emergency planner validates goals, prepares light-node state, adds the fake goal sentinel, and subscribes to world changes.
Initial computation	The planner computes the global flow directions for all reachable cells.
Periodic update	At the configured tick rate, environment objects are refreshed, changed cells are applied, the D* Lite state is repaired, and flow snapshots are published when requested.
Signaling update	Light nodes are updated when cost changes exceed the lighting hysteresis and a dominant direction is selected from local neighborhood directions.
Stop	The dispatcher closes the emergency quit channel, waits for completion, removes preferred-route masks, and marks the emergency as stopped.

Table 1.15: Emergency routing lifecycle.

1.4.2 Flow-Field State

The emergency planner reuses D* Lite ideas, but its output is a direction per cell rather than one agent route. The fake goal connects all exits to a single sentinel, making every reachable cell point toward a suitable exit. Directions include eight planar directions and portal transitions `DIR_IN` and `DIR_OUT`.

State	Role
Fake goal	Sentinel outside the real grid used as a common target for all emergency exits.
Direction map	Per-floor arrays of `domain.Direction` values sent through `FlowSub`.
World subscription	Tracks changed cells and queues signaling nodes whose local flow may need refreshing.
`flowPublisher`	Single publisher for emergency flow snapshots. A new subscription replaces the previous publisher.

Table 1.16: Main state of the emergency flow field.

1.4.3 Environment Updates and Signaling

Before computing or updating the flow field, `applyEnvironmentObjects` queries all oriented objects and refreshes object layers for all floors. Each floor uses the region linked to that layer, clears previous object costs, rasterizes relevant objects, and calls `UpdateFinal`. Changed cells are registered by the world's subscription.

Parameter or dependency	**Emergency effect**
`LightingHystheresis`	Minimum cost-change threshold before a signaling node direction is updated.
`PriorityPathCellsWidth`	Width, in grid cells, used to rasterize preferred routes onto the grid.
`AllObjectsOriented`	Source of the objects used to refresh dynamic costs on all floors.
`SignalingSetDirection`	External operation that writes the selected direction into a signaling node.
`updateTicksPerSecond`	Frequency at which the emergency goroutine refreshes the environment, repairs planning state, and publishes changes when needed.

Table 1.17: Configuration and dependencies relevant to emergency routing.

1.4.4 Preferred Routes and Restoration

Preferred routes arrive as per-floor graphs. `ApplyPreferencePaths` searches the waypoints of each edge and rasterizes a line on the corresponding floor grid. This line applies a negative mask to the base layer so the flow field favors those corridors. Malformed edges with missing waypoints are ignored. If an edge omits its weight, the implementation uses the default value `1`.

When the emergency stops, `RemovePreferencePaths` calls `ResetBase` on every floor to restore normal traversable-space cost while keeping object layers current. Restoration must happen both on ordinary stop and during service shutdown.

Responsibility	Code path
Apply preferred routes	The start handler calls `ApplyPreferencePaths` before creating the flow field.
Remove routes after startup error	If goal conversion or initialization fails, the handler removes preference masks before returning the error.
Remove routes on stop	`completeEmergencyStopLocked` calls `RemovePreferencePaths` and marks the emergency stopped.
Remove routes during shutdown	`stopCurrentEmergency` joins the emergency goroutine and performs the same cleanup even if shutdown races with a client stop request.

Table 1.18: Emergency cleanup responsibilities.

1.4.5 Validation of Emergency Changes

Emergency changes should cover startup, blocked goals, simultaneous start rejection, flow publication, object updates, signaling, preferred routes, and ordered stop. If a change affects signaling, include tests for hysteresis, direction selection, and missing or invalid nodes.

1.5 Public API, NATS Subjects, and Dispatcher

The public API has two equivalent views: typed Go wrappers and raw NATS messages. The wrappers live in `pkg/pathfinding` and `pkg/emergency`; the raw subject names live in `pkg/subjects`. The dispatcher owns the NATS subscriptions and translates messages into internal grid, world, communicator, and flow-field operations.

1.5.1 Boundary Between Public Clients and Internal Handlers

Public operation	Internal payload or handler	Internal effect
`Pathfinding.AgentFindPath`	`AgentFindPathParams`	Converts goal, creates an agent communicator, and registers it in the dispatcher.
`Pathfinding.AgentNaivePathCost`	`AgentNaivePathCostParams`	Computes cost without creating movement state.
`Pathfinding.AgentPathSub`	`IdentifierParams` and `snapshotstream.Envelope`	Streams route cell states through the request reply subject.
`Emergency.Start`	`EmergencyPathParams`	Applies preferred routes, validates goals, starts the flow field, and updates emergency lifecycle state.
`Emergency.FlowSub`	`snapshotstream.Envelope`	Streams emergency direction maps per floor.

Table 1.19: Public operations and internal dispatcher paths.

1.5.2 Subject Registration and Concurrency

`app.Run` subscribes all public subjects. Most handlers execute directly on NATS callbacks. `AgentFindPath`, `BlockingWait`, and `MoveFMeters` are wrapped by bounded worker handlers because they can block long enough to occupy the callback path.

Subject group	Worker limit	Reason
`pathfinding.agent_find_path`	`findpath-handlerworkers`	Route initialization can block while preparing the pathfinding run.
`agent_comm.blocking_wait`	`blockingwait-handlerworkers`	The request waits until an agent completes.
`agent_comm.move_f_meters`	`movefmeters-handlerworkers`	The request waits until a distance is consumed or the route ends.
Other request subjects	No extra worker wrapper	They are expected to finish quickly or stream through a separate goroutine.

Table 1.20: NATS handler concurrency model.

1.5.3 Agent Session Lifecycle

A new `AgentFindPath` call for an agent replaces the previous run for that same identifier. The old communicator is terminated. Completed results are retained for a short interval

so IsMoving, BlockingWait, and ExitError can still respond after the active communicator has left the dispatcher table. During shutdown, stopActiveAgents prevents new registrations, terminates all active communicators, waits for them, and clears the tables.

1.5.4 Subscription Flows

Path and flow subscriptions use request/reply setup but stream multiple messages to the reply subject. The handler returns immediately and a goroutine owns the reply stream. Messages contain done=false while data continues and a final done=true message before the channel closes.

Small snapshots keep the legacy envelope:

```
{"done": false, "cells": {"0": [0, 1, 4]}}
```

Large snapshots use the shared pkg/snapshotstream codec. The service serializes the per-floor map, compresses it with gzip, and publishes a sequence of chunk envelopes:

```
{
  "encoding": "gzip+json",
  "snapshotId": "42",
  "chunkIndex": 0,
  "chunkCount": 3,
  "data": "..."
}
```

chunkIndex is zero-based within one snapshotId. Because data is a Go []byte encoded as JSON, non-Go clients must first base64-decode that field for each chunk. Consumers then join all chunks for the same snapshot, decompress the combined payload, and decode the JSON as the same per-floor map otherwise carried by cells. The Go wrappers use snapshotstream.Assembler and expose only reconstructed snapshots on AgentPathSub and FlowSub. Service handlers should use snapshotstream.MarshalSnapshot for data messages and snapshotstream.MarshalDone for terminal messages.

Replacing an agent path publisher closes the previous publisher for that communicator. Replacing the emergency flow publisher closes the previous global flow publisher. Clients must be ready for stream closure and should resubscribe if they need to become the active observer again.

1.5.5 Public Errors

Domain errors are registered with the shared message package so raw NATS clients can receive stable error codes. Go wrappers additionally wrap failures with operation and NATS-phase sentinels such as ErrNATSRequest, ErrNATSResponse, ErrNATSSubscription, and ErrNATSPublish.

Public error	Typical situation
`ErrAgentCommNotFound`	A communicator operation references an identifier without an active or retained session.
`ErrAgentNoPath`	No traversable route exists from the current agent position to the goal.
`ErrAgentTerminated`	The route was explicitly terminated.
`ErrAgentStillRunning`	`ExitError` was queried while the route is still active.
`ErrAgentExitedWithMetersLeft`	A meter movement request could not consume the full requested distance because the route ended first.
`ErrEmergencyStillRunning`	An emergency start was requested while another emergency was active or stopping.
`ErrDispatcherShuttingDown`	The dispatcher is draining and no longer accepts new agent registrations.

Table 1.21: Domain errors that are part of the public API.

1.5.6 Validation of Public API Changes

When adding or changing public API behavior, update all related contracts together: subject constants, payload type, dispatcher handler, Go wrapper, integration tests, and user manual. Include malformed-payload or missing-session cases for handlers that accept external input.

1.6 Configuration and Embedded Execution

Configuration affects both the standalone executable and embedded execution. It defines NATS connectivity, logging, grid shape, layers, portals, handler worker limits, geotruth pool sizes, and algorithm parameters.

1.6.1 Configuration Sources

`main.go` calls `LoadConfig(nil)`. This creates a Viper instance, registers defaults from `DefaultConfig`, loads `config/config.yaml`, and allows `PTHFSERVICE_*` environment variables to override keys. Tests, tools, and benchmarks can pass an explicit YAML path.

Source	Precedence	Use
`DefaultConfig`	Lowest	Provides local defaults for NATS, grid, maps, algorithm parameters, and worker limits.
`config/config.yaml` or explicit path	Middle	Project or scenario configuration.
`PTHFSERVICE_*`	Highest	Operational override. Dotted keys become underscores, for example `PTHFSERVICE_APP_SERVERURL`.

Table 1.22: Configuration sources and precedence.

1.6.2 Public Configuration Structure

Type	Responsibility
`AppConfig`	NATS URL and logging destinations.
`GridConfig`	Layers, portals, rows, columns, pixels per cell, meters per cell, and wall aura.
`PathfindingConfig`	Shared free-movement height plus the agent, emergency, and geotruth blocks.
`AgentConfig`	Vision radius, real-position update cadence, reusable-world limit, sparse value storage, NATS worker limits, and debug output.
`EmergencyConfig`	Signaling hysteresis, preferred-route width in cells, and debug output.
`GeotruthConfig`	Internal NATS connection pool sizes for geotruth queries and publishes.

Table 1.23: Public configuration groups.

1.6.3 Dependency Injection

Field	Interface	Purpose
`Ex`	`ExternalSystem`	Object traversal costs, floor heights, and emergency signaling operations.
`Qu`	`GeoQuery`	Object data, local vision, all oriented objects, and floor/region lookup.
`Pu`	`GeoPublish`	Movement publication and commit acknowledgement for real object positions.

Table 1.24: Injected dependencies and subsystem use.

`DefaultDependencies` is useful for local runs and tests because it creates geotruth NATS pools and a simulated `ExternalSystem`. In a real installation, the integrator should provide implementations that match the deployment's geotruth and signaling services.

1.6.4 Standalone and Embedded Execution

The standalone executable and embedded API converge in `app.Run`. In standalone mode, `main.go` loads the file, creates default dependencies, and waits for OS signals. In embedded mode, the caller constructs `Config`, supplies dependencies, calls `embedded.Run`, and later calls `Shutdown` on the returned dispatcher.

Responsibility	**Standalone**	**Embedded**
Configuration	`LoadConfig(nil)` from `config/config.yaml`.	Caller can use defaults, explicit load, or construct a configuration.
Dependencies	`embedded.DefaultDependencies`.	Caller should usually provide production dependencies.
Lifecycle	OS signal triggers shutdown.	Caller owns cancellation and must call `Shutdown`.

Table 1.25: Standalone and embedded execution responsibilities.

1.6.5 Validation of Configuration Changes

For a new configuration key, add the public field, `mapstructure` tag, default value, documentation, and tests for defaults, YAML loading, and environment override. If the value affects runtime behavior, add a functional test in the subsystem that consumes it.

1.7 External Integrations and Test Doubles

The pathfinding service depends on external systems but keeps them behind small interfaces. Internal packages should not create real external clients by themselves. They receive dependencies prepared by the standalone startup, embedded API, or tests.

1.7.1 Dependency Contracts

Contract	Primary users	Notes
`ExternalSystem`	Agent movement, emergency signaling, object costs.	Must propagate errors with enough context to diagnose external failures.
`GeoQuery`	Coordinate conversion, agent local vision, emergency object refresh.	Region names returned by `RegionFromPoint` must match configured layer names.
`GeoPublish`	Agent real-position publishing.	Returns a commit acknowledgement. Resource ownership belongs to the concrete implementation or the pool that wraps it.

Table 1.26: External dependency contracts.

1.7.2 Use by Subsystem

Agent planning uses `GeoQuery.ObjectData`, `RegionFromPoint`, `NearbyObjectsOf`, `ObjectTraversalCost`, `HeightAtFloorPoint`, and `GeoPublish.UpdateObjectPosition`. Emergency routing uses `AllObjectsOriented`, `LightNodeRegex`, and `SignalingSetDirection`. Startup resolves the available signaling nodes once when the dispatcher is created.

1.7.3 Embedded Execution and Real Services

Embedded execution can be used with the default dependencies or with real services. The default dependency path still connects to NATS and creates geotruth query/publish pools, but it uses a simulated external system for domain costs and signaling. For production, provide dependencies that implement the public interfaces and close any external resources they own.

1.7.4 Test Doubles

`internal/mock` provides test doubles for the domain and geotruth parts needed by the service. They are used for unit, component, integration, and performance tests where deterministic behavior is more important than full external-system fidelity.

Double	Simulated behavior	Main use
`mock.GeoTruth`	In-memory object store, lookup by ID, local object queries, and oriented object sets.	Agent and emergency tests.
`mock.ExternalSystem`	Object traversal costs, floor heights, and signaling methods.	Algorithm and integration tests.
`internal/pathfinding/testing` scenarios	Reusable map and route cases.	Black-box pathfinding behavior.

Table 1.27: Main test doubles and scenarios.

1.8 System Extension and Technical Limits

The system supports local changes, but some resources are process-wide: configuration, dependencies, and the global world. An extension must not initialize unrelated worlds or modify global configuration while agents or emergencies are active unless it also defines and tests a new synchronization strategy.

1.8.1 Extension Points

Extension	Contracts to update	Minimum validation
New NATS subject or public operation	`pkg/subjects`, dispatcher payload, handler registration, Go wrapper, and user manual.	Integration test, invalid-payload case, and public-error review.
Snapshot stream envelope	`pkg/snapshotstream`, dispatcher publication helpers, Go stream assemblers, raw-NATS documentation, and user manual.	Codec tests plus public `AgentPathSub` and `FlowSub` round trips for chunked snapshots.
New agent communicator command	Public method, subject, handler, internal communicator method, and behavior for finished or missing agents.	Tests with active agent, terminated agent, cancellation, and post-completion calls.
New configuration parameter	Public field, `mapstructure` tag, default, file/env loading, and documentation.	Configuration-load test and subsystem functional test.
New external dependency	Public interface method, real implementation, test double, and subsystem call.	Double-based test, error propagation test, and integration case when NATS or geotruth is involved.
New algorithm or search variant	Internal input boundary, state types, global-grid interaction, and result publication.	Algorithm unit tests, no-path cases, multi-floor cases if applicable, and public API test when externally observable.

Table 1.28: Main extension points and validation requirements.

1.8.2 Current Technical Limits

Limit	Current behavior	Implication
Process-wide world	The global world and layer translation are shared.	Multiple independent worlds in one process require redesign.
Static signaling set	Light nodes are resolved when the dispatcher starts.	Nodes appearing or disappearing during an emergency are future work.
Grid-based movement	Public coordinates are meters, but algorithms operate on rasterized cells.	Precision and policy rules depend on grid resolution.
Preferred-route restoration	Emergency preferred routes modify global base costs and are reset on stop.	Unexpected shutdown paths must preserve cleanup.
Per-agent local world	Agents copy or link local dynamic state per route.	Large maps and many agents may require paging or higher-level guidance.

Table 1.29: Current technical limits and their design intent.

1.8.3 Checklist Before Accepting an Extension

Before accepting a shared change, identify whether it affects configuration, grid/rasterization, algorithms, public API, external integrations, or lifecycle. Then confirm documentation, tests, and public wrappers are updated together. If a limitation remains, document it here or in the project conclusions rather than letting it remain implicit.

1.9 Tests, Benchmarks, and Technical Validation

The repository uses Go's `testing` package. Tests live beside the packages they exercise, with additional integration-style tests under `internal/integration` and shared behavior scenarios under `internal/pathfinding/testing`. Heavy diagnostics and benchmarks are kept separate from ordinary tests.

1.9.1 Test Map by Responsibility

Area	Typical packages	Purpose
Grid and raster	`internal/grid`, `internal/raster`, `pkg/tuning`	Coordinate conversion, cost layers, image rasterization, wall aura, portals, subscriptions, and visualization helpers.
Agent algorithm	`internal/pathfinding/agent`	D* Lite behavior, local changes, movement, communicator commands, and multi-floor paths.
Emergency algorithm	`internal/pathfinding/emergency`	Flow-field planning, preferred routes, signaling, environment updates, and stop cleanup.
Public API	`pkg/pathfinding`, `pkg/emergency`, `internal/app`, `internal/integration`	NATS request/response behavior, wrappers, errors, lifecycle, and shutdown.
Configuration and embedded	`internal/config`, `embedded`, integration tests	Default values, YAML loading, environment overrides, embedded startup, and dependency cleanup.

Table 1.30: Test responsibilities by subsystem.

1.9.2 Unit and Component Tests

Unit tests validate small, deterministic behavior: priority queues, raster primitives, grid cost math, coordinate conversion, portal lookup, paged value storage, and public wrapper error classification. Component tests exercise larger behavior such as D* Lite replanning, agent communicators, flow-field updates, and dispatcher shutdown.

1.9.3 API Boundary Error Tests

The API tests include representative malformed JSON, missing communicator, dispatcher-shutdown, emergency-already-running, idempotent stop, and post-agent completion cases. The goal is scenario-driven validation, not duplicating every malformed payload variant for every subject.

1.9.4 Public API Integration Tests

Integration tests start NATS and the service path, then use public clients or raw subjects to validate observable behavior. They check direct routes, multi-floor routes, subscrip-

tions, movement commands, emergency start/stop, and error propagation through the same public surface that external consumers use.

1.9.5 Benchmarks and Heavy Tests

Benchmarks are used for diagnosis and scalability evidence. Ordinary benchmarks measure local structures or algorithms. Heavy benchmarks require explicit environment variables and should be run only when evaluating capacity or full-stack behavior.

Benchmark group	Focus	Use
Grid/raster benchmarks	Map loading, portal lookup, world creation.	After changing rasterization or world data structures.
Agent benchmarks	D* Lite planning, local cost updates, movement capacity.	After changing agent planning, movement cadence, or virtual worlds.
Emergency benchmarks	Flow-field planning, object refresh, preferred-route application.	After changing emergency recomputation or signaling behavior.
API capacity benchmarks	NATS wrappers, handler workers, geotruth pool settings.	When evaluating full-stack concurrency.

Table 1.31: Benchmark groups and when to use them.

Typical heavy benchmark commands include:

```
$env:PATHFINDING_BENCH_HEAVY='1'; go test ./internal/performance -run ^$ -bench
↪  BenchmarkSynthetic_AgentCapacity -benchtime=1x -count=7 -benchmem
$env:PATHFINDING_BENCH_HEAVY='1'; $env:PATHFINDING_BENCH_REAL='1'; go test
↪  ./internal/performance -run ^$ -bench BenchmarkReal_AgentCapacity
↪  -benchtime=1x -count=7 -benchmem
$env:PATHFINDING_BENCH_HEAVY='1'; go test ./internal/pathfinding/emergency -run
↪  ^$ -bench BenchmarkFlowfield_MovingObjectsCapacity -benchtime=1x -count=7
↪  -benchmem
```

When using `-count=7`, summarize latency with the median and treat a point as stable only when all seven runs satisfy the benchmark criterion. Partial success should be described as unstable, not as guaranteed capacity.

Agent capacity benchmarks distinguish `RawStep`, `ExternalTick`, and `RealTime`. `ExternalTick` best represents simulations whose clock is controlled by another system: agents start stopped, external ticks ask each active agent to move a meter distance, and leftover distance is accumulated. `RealTime` measures autonomous movement where the service controls cadence.

Emergency moving-object capacity measures a synchronous emergency tick: read oriented objects, rasterize them, apply changed cells, and repair the flow field. The main

axis is `moving_objects`; `p95_tick_ms` is compared with the target tick budget, while `missed_ticks` remains diagnostic.

1.9.6 Coverage, Execution, and Change Validation

The main validation command is:

```
make test
```

This target runs `go test` with verbose output, local package coverage, and `coverage.out`. Local coverage shows what each package's own tests exercise, but it does not attribute code executed by tests from other packages. For complete cross-package coverage, use:

```
make test-cover-full
```

This target runs all module tests with `-coverpkg=./...` and produces `coverage-full.out`. Standard Go coverage tools can open both profiles.

Validation type	**Command**	**When to use it**
Full tests	`make test`	Before closing changes to algorithms, grid, API, configuration, or lifecycle.
Full coverage	`make test-cover-full`	When measuring all code exercised by any test in the module.
Specific package	`go test ./internal/...` or a specific package	During local iteration on a small area.
Local coverage HTML	`make cover-html`	After `make test`.
Cross-package coverage HTML	`make cover-full-html`	After `make test-cover-full`.
Benchmarks	`go test -bench=.` on the affected package	When changing data structures, rasterization, planning, or resource reuse.

Table 1.32: Recommended technical validation commands.

1.10 Maintenance, Diagnosis, and Operational Problems

Maintenance requires observing logs, external resources, and active route lifecycle together. Most visible API errors come from inconsistent configuration, unavailable NATS or geotruth, coordinates that do not map to the grid, missing layers, disconnected routes, or incomplete cleanup during stop.

1.10.1 Logs and Diagnosis

Internal logging uses a `[PATHFINDING]` prefix, date, time, and source file. Configuration chooses standard error, a file, both, or neither. Disabling all output can be useful for tests but makes production diagnosis difficult.

Area	Observable signal	First check
Startup	Panic or error before subjects are registered.	Configuration, map paths, NATS URL, and default dependencies.
Rasterization	Complex-image warning or failure opening a map path.	Image path relative to the execution directory and configured dimensions.
Portals	Portal dropped because of invalid cost, endpoint, or same layer.	Cost, source/destination layers, and meter coordinates.
NATS API	Client timeout or no response.	Same NATS server, registered subject, and service startup logs.
Agents	Communicator error, no path, or real-position update failure.	Agent existence in geotruth, valid goal floor, connectivity, and publisher acknowledgement.
Emergency	Start failure, unchanged flow, or no signaling update.	Goals, preferred routes, light nodes, and oriented-object data.

Table 1.33: Initial diagnosis points by symptom.

1.10.2 Graceful Shutdown

Normal shutdown enters through `Shutdown`. In the standalone executable, this follows `SIGINT` or `SIGTERM`. In embedded mode, the caller must invoke it. The order matters because agent goroutines, emergency goroutines, publishers, and NATS subscriptions may be active simultaneously.

Phase	Resource	Risk if it fails
Stop agents	Active communicators and retained results.	Blocked goroutines, movements after shutdown, or lost terminal results.
Stop emergency	Quit channel, done channel, preferred routes.	Base costs not restored or flow publisher left open.
Drain NATS	Dispatcher subscriptions and pending replies.	Clients miss final replies or callbacks run while state is being released.
Cancel context	Root service context.	Internal operations wait on already closed resources.
Close external resources	Connections, clients, or descriptors owned by dependencies.	Resource leaks in the embedding process.

Table 1.34: Relevant phases of graceful shutdown.

1.10.3 Common Problems

Problem	Likely cause	Initial action
Timeout on a public call	NATS unreachable, subject not registered, or service not started.	Check URL, service startup, and subject registration logs.
Agent not found	Identifier missing in geotruth or completed result expired.	Verify object registration and distinguish agent ID from active session state.
No route	Goal outside grid, missing layer, walls without portals, or unreachable cost.	Check converted coordinates, layers, portals, and rasterized map.
Movement without real update	Publishing dependency failure or unsuitable update cadence.	Check `GeoPublish`, commit acknowledgement, floor height, and update configuration.
Emergency does not start	Another emergency is active, goal is invalid, or flow creation fails.	Check dispatcher emergency state, goals, and preferred-route restoration on failure.
Signaling does not change	Light nodes not resolved, hysteresis too high, or dominant direction unchanged.	Check node query, computed directions, and hysteresis.

Table 1.35: Common operational problems and first technical action.

1.10.4 Maintenance Checklist

Before changing a shared area, identify whether the change affects configuration, grid, algorithm, public API, or lifecycle. Then review external contracts and run the minimum validation for the affected subsystem.

Review	**Criterion**
Effective configuration	The changed key has a default, file loading behavior, and documented effect.
External resources	Queries, publications, and external-system calls have timeouts and contextual errors.
Resource ownership	The owner of subscriptions, channels, worlds, publishers, and connections is clear.
Public compatibility	If an error, subject, JSON field, or public type changes, update the user manual and integration tests.
Diagnostic logs	Logs help locate failure phase without leaking sensitive data or flooding normal output.
Tests and benchmarks	Run subsystem validation and compare benchmarks before and after performance changes.

Table 1.36: Technical checklist before closing maintenance changes.

1.10.5 Validation After Incidents

A fixed incident should leave a reproducible test or check. Configuration failures need loading or invalid-value tests. Concurrency failures need tests for the observed race. Integration failures should exercise the public path that failed.

Incident	**Recommended validation**
Startup error	Configuration or embedded-start test, plus logging check if the failure phase was unclear.
NATS timeout or error	Application or integration test confirming subject registration, response, and shutdown without pending goroutines.
Incorrect route	Algorithm unit test and integration test if the difference was visible through the public API.
Contaminated global state	Reuse or cleanup test for virtual worlds, local portals, preferred routes, or completed results.
Shutdown error	`Shutdown` test with active agents, active emergency, or simultaneous client stop.
Performance regression	Benchmark of the affected component with the same map and configuration before and after the correction.

Table 1.37: Recommended validation after operational incidents.

Bibliography

[1] S. Koenig and M. Likhachev, "D* Lite", in *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI-02)*, Edmonton, Alberta, Canada, 2002, pp. 476–483. Available: `https://cdn.aaai.org/AAAI/2002/AAAI02-072.pdf`. Accessed: June 15, 2026.

[2] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice in C*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1995.

[3] J. Pineda, "A Parallel Algorithm for Polygon Rasterization", *ACM SIGGRAPH Computer Graphics*, vol. 22, no. 4, pp. 17–20, 1988, doi: 10.1145/54852.378457.